Math 339: Mechanics

The Final Chapters: The Duffing Oscillator

Willamette University, Fall 2020

The duffing oscillator[1] is a type of driven dampened oscillator with some non-linear restoring force; rather than being linearly proportional to some constant k (obeying Hooke's Law), the duffing oscillator is characterized by non-linear term. Modifications to these types of oscillators can have potentially drastic implications over complex circuit interactions, buckling fores in rigid columns, or even modeling complex springs, whose long-term behavior is erratic and non-periodic (chaotic, if you will). We will attempt to develop an understanding of how this oscillator behaves under both undriven and driven conditions, how sensitive it is toward initial conditions, and how to approach modeling chaotic behavior that, ordinarily, might seem impossible to even approach.

A VERY BASIC BACKGROUND

In order to provide for a seamless transition from problem to problem, any necessary background information (before approaching a particular problem) will be denoted as a "detour". This will avoid any flipping back and forth for the reader[1], and, in case some readers are already familiar with this background information, can easily (and effortlessly) gloss over it.

Most of this very basic background actually just comes from the statement of the problem/Professor Rembold. All I do here is elaborate on the responsibility each of these parameters have. The other, majority, of the background will be provided as "detours" as we go.

The Duffing equation[2] springs up (oh the puns!) in a wide variety of physics and engineering problems, as it describes a damped oscillator with some non-linear spring response. In particular, variations on the equation can describe complex circuit interactions, buckling forces in rigid columns, or spring oscillations for springs which do not obey Hooke's law. The equation for a driven oscillator looks like:

$$\ddot{x} + \beta\dot{x} + \alpha x + \gamma x^3 = Fcos(\omega t) \tag{1}$$

where the x^3 term (aka the γ parameter) is that non-linear spring response/restoring force[3]. Additionally, here, the β serves here as the damping force, the α the spring constant, ω the angular frequency, and F the amplitude of the driving force. As we will see throughout working through the below problem, the duffing equation is highly sensitive to given initial conditions.

THE PROBLEM

Here we'll work with the parameters of:

$$\beta = 0.05$$

$$\alpha = -3$$

$$\gamma = 1$$

(a) Initially we'll look at the non-driven system, so let $F = 0$.

i. Determine the location of the equilibrium points.

ii. To see if these points are stable or unstable, plot a phase space diagram of the solution for an object that initially had $x_0 = 0.1$, and $\dot{x_0} = 0$. Do your equilibrium points look like they'd be stable or unstable?

---

[1] Hi. Please do not be frightened by the length of this paper. Most of it is actually coding/figures, so need not be afraid!

[2] As part of being a math major, I really like explicitly stating what we are given/what we know for sure. Thus, I proceed to explicitly restate the problem, and then restate it (as well as any suppositions being made) as we go.

[3] I've actually only ever heard "restoring force", but, yet again, that was because I had Professor McNicholas as a teacher for Differential Equations...

iii. Plot a quiver plot overtop of your plotted solution above. This will help you see the general movement and path that an object would follow for a variety of initial conditions.

iv. Matplotlib offers a nicer way to see these streamlines with a streamplot. Create one covering the same region as your above plot to see how they compare.

(b) Now we'll look at the driven system. Let's set

$$F = 8$$

$$\omega = 2$$

i. Solve the differential equation numerically for two different initial conditions, one with $x_1 = 2$, $\dot{x_1} = 0$ and the other with $x_2 = 2.001$ and $\dot{x_2} = 0$, over about 200 seconds. Plot the phase space of both solutions. Does this motion seem chaotic?

ii. To check, make a plot of the time vs the natural log of the absolute value of the difference between the two solutions. Does the long term behavior seem to be chaotic or not? You could compute a Liapunov exponent from this plot if you wanted.

(c) Repeat part (b) but now with $\omega = 1.2$.

(d) Create a Poincare section for the same parameters as given in part (c). This is mostly the same process as finding the solutions like you have been above, but you'll want to look at the time only once every period (so adjust your step size in times accordingly). In addition, you'll want to plot the phase space as points rather than lines. The minimum time to run the solution over to see a decent strange attractor is 10000 periods. Be patient, it will take a bit of time to process this. Moreover, since you only want the long term motion, you should really ignore the first 500 or so iterations.

NON-DRIVEN SYSTEM

(a) **Initially we'll look at the non-driven system, so let $F = 0$.**

i. **Determine the location of the equilibrium points.**

Recall (1),

$$\ddot{x} + \beta\dot{x} + \alpha x + \gamma x^3 = Fcos(\omega t)$$

In order to find the location of equilibrium points[4], we[5] can use (1), plug in our given parameters ($\beta = 0.05$, $\alpha = -3$, $\gamma = 1$) in the case of the non-driven system ($F = 0$), and use the fact that, when we find equilibrium positions, we are finding solutions to the equation for x when $\dot{x} = 0$ and $\ddot{x} = 0$; that is, for equilibrium positions we are looking for solutions to the equation where an object is at rest ($\dot{x} = 0$) and it will stay at rest ($\ddot{x} = 0$). Thus, plugging these in and solving for solutions for x,

$$3x + x^3 = 0 \tag{2}$$

$$x(-3 + x^2) = 0 \tag{3}$$

Notice one equilibrium solution happens when $x = 0$, and the other occurs when the second term is zero; that is,

$$-3 + x^2 = 0 \tag{4}$$

---

[4] Rembold, J. (2020, November 20). Mechanics: Lecture 26 [Video]. YouTube.

[5] Yes, I did just cite the latest mechanics video (because we did it there), but, I mean, we've also been doing this all semester long (since chapter 4!)...

$$x^2 = 3 \tag{5}$$

$$x = \pm\sqrt{3} \tag{6}$$

Therefore, our equilibrium points will happen at the points when $x = 0$, $x = \sqrt{3}$, and $x = -\sqrt{3}$.

ii. **To see if these points are stable or unstable, plot a phase space diagram of the solution for an object that initially had $x_0 = 0.1$, and $\dot{x_0} = 0$. Do your equilibrium points look like they'd be stable or unstable?**

A Detour: Phase Space Diagrams Simply put[6], a phase space diagram[7] is a diagram that plots position (x) vs. velocity ($\dot{x}$). As time passes, the points that fill this diagram, with particular x's and $\dot{x}$'s, trace out a curve, which is called a **phase space** trajectory (or, alternatively, a space-state orbit). This means that, as t advances, the curve is traced; this gives us a hint and foreshadows the utility of quiver plots when using these kind of diagrams. ■

Of course, before we even begin to plot the phase-space diagram, we have run into a conundrum: how do we plot it? The duffing equation (1) gives us a second order non-linear ordinary differential equation, but, if we want to be able to solve the phase space diagram, we need a first order one!

Recall our method of using Scipy's ODEInt that we have throughout the semester. If you do not recall, or are particularly bad at it[8], I will walk through the steps regardless. First of all, recall (1),

$$\ddot{x} + \beta\dot{x} + \alpha x + \gamma x^3 = Fcos(\omega t)$$

Let us solve for $\ddot{x}$, as well as writing these time derivatives more like being a time derivative, before we make any substitutions. We will proceed by using a series of algebraic manipulation and the definition of first and second derivatives:

$$\ddot{x} + \beta\dot{x} + \alpha x + \gamma x^3 = 0$$

$$\ddot{x} = -\beta\dot{x} - \alpha x - \gamma x^3 \tag{7}$$

$$\frac{d^2x}{dt^2} = -\beta\frac{dx}{dt} - \alpha x - \gamma x^3 \tag{8}$$

Thus, making the following substitutions,

$$\frac{dx}{dt} = v \tag{9}$$

$$\frac{d^2x}{dt^2} = \frac{dv}{dt} \tag{10}$$

We end up with

$$\frac{dv}{dt} = -\beta\frac{dx}{dt} - \alpha x - \gamma x^3 \tag{11}$$

---

[6] Whoops! I typically reserve this notation for denoting the beginning of a proof (and, ironically, the black square for the end of a proof), but I thought it looked pretty here/would be easier to the reader. As you were

[7] Taylor, J. R. (2005). Classical Mechanics. University Science Book

[8] Like I am (whoops)

$$\frac{dv}{dt} = -\beta v - \alpha x - \gamma x^3 \tag{12}$$

which we *can* plug into Scipy's ODEInt to solve, as can be seen in Figure 1. This code generates the Phase Space Diagram, as seen in Figure 2.

```
#ODEInt~ :)
def DE(state,ts):
    derivs = np.zeros(2)
    derivs[0] = state[1] #xs
    derivs[1] = -(beta*(state[1]))-(alpha*state[0])-(gamma*(state[0]**3)) #dot{x}
    return derivs
```

Figure 1: Python code, utilizing Scipy's ODEInt, that was used in order to generate the phase space diagram for the undriven dampened oscillator.

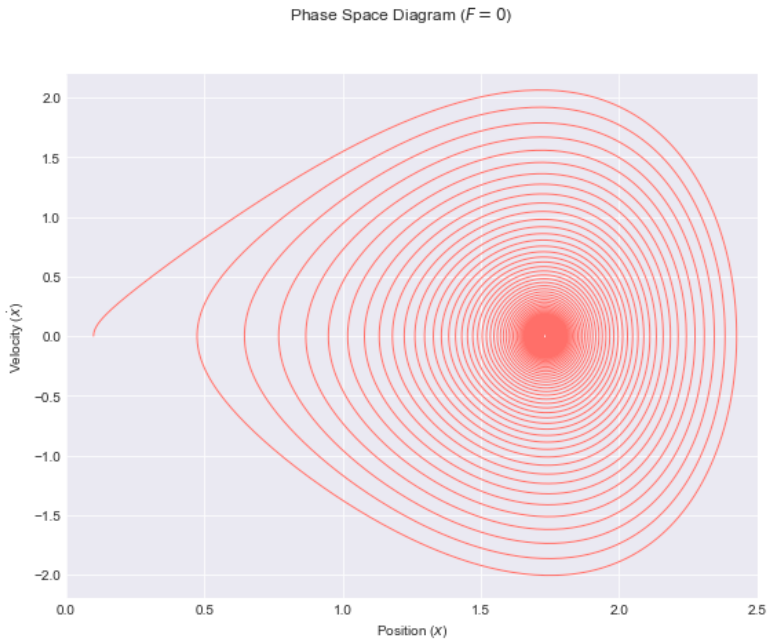

Figure 2: Phase Space Diagram for an undriven dampened oscillator, with a non-linear restoring force, with the given parameters $\beta = 0.05$, $\alpha = -3$, and $\gamma = 1$.

iii. **Plot a quiver plot overtop of your plotted solution above. This will help you see the general movement and path that an object would follow for a variety of initial conditions.**

In order to plot a quiver plot, on top of our plotted solution taking the form of a phase space diagram, we need functions in terms of how our position x and our velocity v are changing with respect time; that is, in order to find the instantaneous slope at every point on our phase-space diagram, we need functions (in terms of these points) in order to do so. Thus, in the code as can be seen in Figure 3, let us create a range of x's and v's as already plotted in our phase-space diagram; in particular[9], let our x's range from 0 to 2.5 and let our v's range from -2.2 to 2.5. Using these ranges, we can create our traditional mesh grid (with one particular point being given by a particular x and a

[9] It would have been more efficient to let our xs range from the max and min to the xs we plotted above, but I wanted to make sure (1) we captured everything and (2) that it would look pretty; thus, my range that I have coded is a bit larger than both of the ranges given in the phase-space diagram.

particular v, taking the form of (x, v)) and define two functions $DX(X, V)$ and $DV(X, V)$ that take in these points, returning the change in position and change in velocity respectively, located in Figure 3 and Figure 4.

```
#Creating meshgrid
#Going a little past the points of our grid above
#so that it doesn't look chopped off lol
xs = np.arange(0,2.5,0.15)
vs = np.arange(-2.2,2.5,0.15)
X,V = np.meshgrid(xs,vs)

#Literally just functions of our ODEInt
#so that we can work with meshy grids
def DX(X,V):
    return(V)
def DV(X,V):
    return(-(beta*(V)-(alpha*X)-(gamma*(X**3))))

#Graphing time~
f2 = plt.figure(figsize=(9,7))
#Titling and Coloring :)
plt.suptitle('Quiver Plot over Phase Space Diagram ($F=0$)')
plt.plot(sols[:,0],sols[:,1], lw=2, color='lightpink') #Phase-space
plt.quiver(X[::2],V[::2],DX(X,V)[::2],DV(X,V)[::2],color='maroon') #Quiver plot
plt.xlim(0,2.5)
plt.ylim(-2.2,2.2)
plt.xlabel('Position ($x$)')
plt.ylabel('Velocity ($\dot{x}$)')
plt.show()
```

Figure 3: Python code that plots a quiver plot overtop the undriven phase space diagram, as plotted before. Utilizes two functions $DX(X, V)$ and $DV(X, V)$ that, as given in their names, are in terms of the points on the grid that are given as an ordered pair of a particular (x, v) in X and V respectively. These functions are the same as we derived in the previous ODEInt code only, in order to generate the Quiver plot, are now in terms of X and V, which are in terms of our generated meshgrid.)

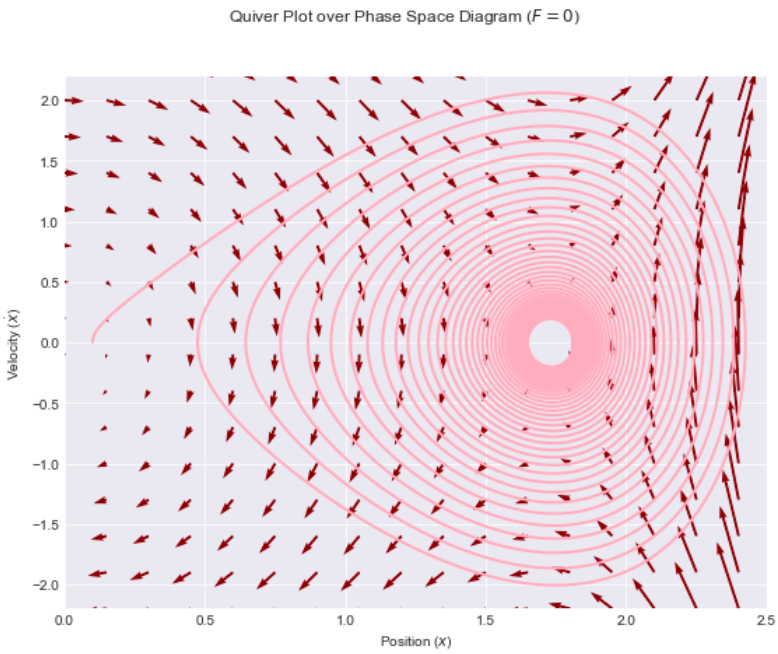

Figure 4: Generated Quiver plot with python code as given from Figure 3.

iv. **Matplotlib offers a nicer way to see these streamlines with a streamplot. Create one covering the same region as your above plot to see how they compare.**

Fortunately, for this part, most of this part's work has already been completed in the previous stem. Since a stream plot is meant to plot the trajectories, as seen in (c), we can plot the same exact things/components, but just used plt.streamplot() as seen in Figure 5 which generates the stream plot in Figure 6.

```
f3 = plt.figure(figsize=(9,7))
#Titling and Coloring :)
plt.suptitle('Phase Space Diagram ($F=0$)')
plt.plot(sols[:,0],sols[:,1], lw=2, color='lightpink')
plt.streamplot(X,V,DX(X,V),DV(X,V),color='maroon')
plt.xlabel('$x$')
plt.ylabel('$v$')
```

Figure 5: Python code plotting the trajectories from the quiver plot in the form of the stream plot, generated as Figure 6.

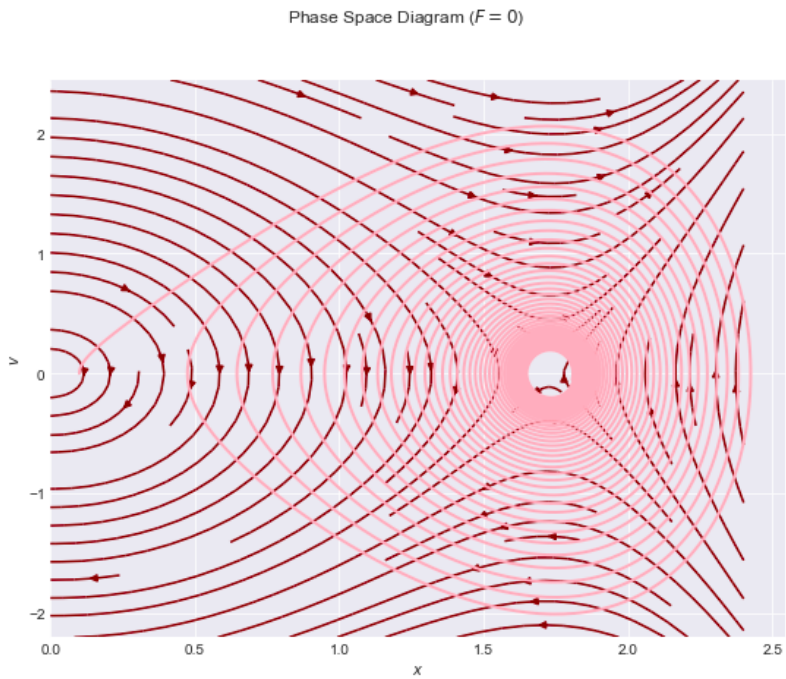

Figure 6: Stream plot from the python code as can be seen in Figure 5.

DRIVEN SYSTEM ($\omega = 2$)

(b) **Now we'll look at the driven system. Let's set**

$$F = 8$$

$$\omega = 2$$

i. **Solve the differential equation numerically for two different initial conditions, one with $x_1 = 2$, $\dot{x_1} = 0$ and the other with $x_2 = 2.001$ and $\dot{x_2} = 0$, over about 200 seconds. Plot the phase space of both solutions. Does this motion seem chaotic?**

Once again, we can do the same process as illustrated in part (a). However, instead of being able to cancel out the driving force component (because there we didn't have a driving force ($F = 0$)), we will have to account for it.

Recall (1),

$$\ddot{x} + \beta\dot{x} + \alpha x + \gamma x^3 = Fcos(\omega t)$$

Thus, using the same exact[10] process as already completed in (a),

$$\ddot{x} + 0.05\dot{x} + (-3)x + x^3 = 8cos(2t) \tag{13}$$

$$\ddot{x} = -0.05\dot{x} - (-3)x + x^3 + 8cos(2t) \tag{14}$$

$$\frac{d^2x}{dt^2} = -0.05\frac{dx}{dt} - (-3)x + x^3 + 8cos(2t) \tag{15}$$

$$\frac{dv}{dt} = -0.05\frac{dx}{dt} - (-3)x + x^3 + 8cos(2t) \tag{16}$$

$$\frac{dv}{dt} = -0.05v - (-3)x + x^3 + 8cos(2t) \tag{17}$$

we can now plot this, using this updated equation (with $\omega = 2$), precisely as we did before, as given in Figure 7. The plots for $x_0 = 2$ and $x_0 = 2.001$ are given in Figures 8 and 9, respectively, with the comparison between the two plots given in Figure 10.

From the motion as displayed in Figures 8-10, it does appear that this behavior is chaotic! If it were not chaotic, we would expect some of these lines to trace over one another or to repeat themselves, but that does not appear to be the case here.

```
#ODEInt~ :)
def DE(state,ts):
    derivs = np.zeros(2)
    derivs[0] = state[1] #xs
    derivs[1] = -(beta*(state[1]))-(alpha*state[0])-(gamma*(state[0]**3))+(F*np.cos(w*ts)) #dot{x}
    return derivs
```

Figure 7: Updated ODEInt solver! This now includes (1) different parameters and (2) our driving force! This one code generates the three plots below, as given in Figure 8, 9, and 10. It does use a very large time interval though, going from 0 to 200 seconds, in order to display the trajectory's potential chaotic nature.

[10] Well, not quite exactly. Just not solving for the equilibrium solutions on this one :P

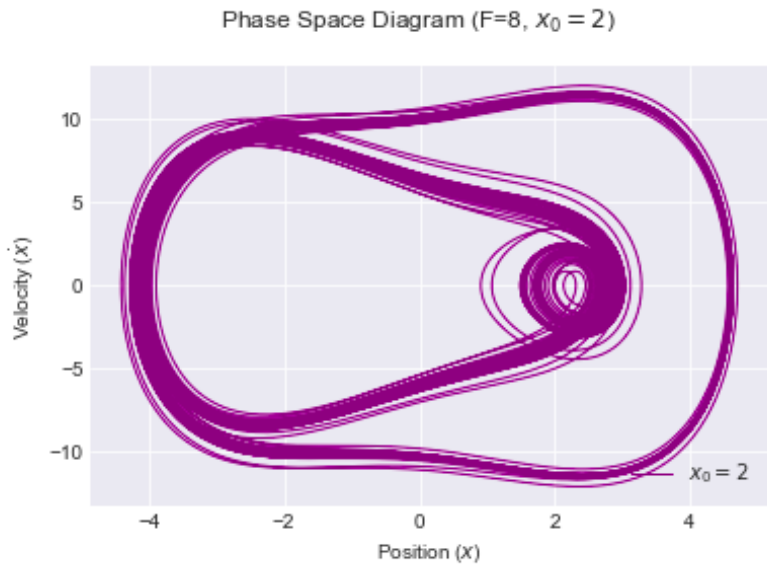

Figure 8: Phase space diagram, with $x_0 = 2$, $\omega = 2$, and $F = 8$.

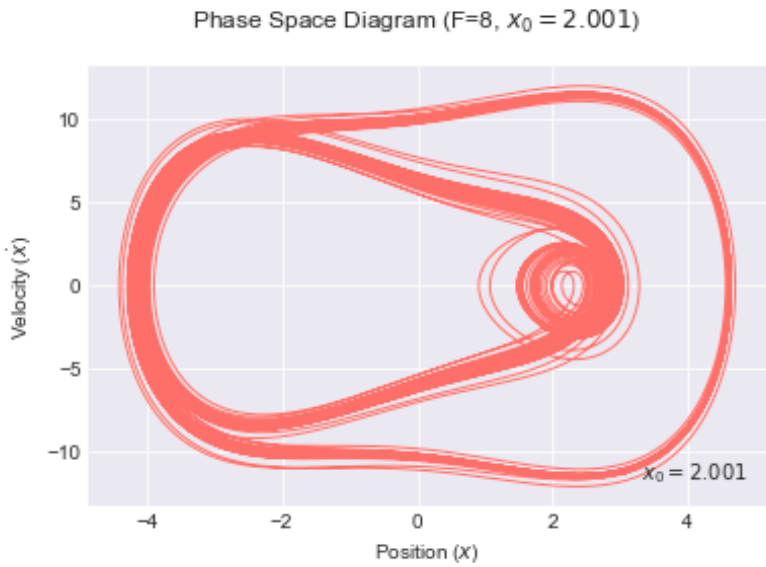

Figure 9: Phase space diagram, with $x_0 = 2.001$, $\omega = 2$, and $F = 8$.

ii. **To check, make a plot of the time vs the natural log of the absolute value of the difference between the two solutions. Does the long term behavior seem to be chaotic or not? You could compute a Liapunov exponent from this plot if you wanted.**

Simply put, if our solutions for $x = 2.001$ are given by sols1, and if our solutions for $x = 2$ are given by sols2, we can plot the natural log of the absolute value of the difference by

$$LN(|sols1 - sols2|) \tag{18}$$

From here, we can plot this into python, as given by Figure 11 and whose output is given by Figure 12.

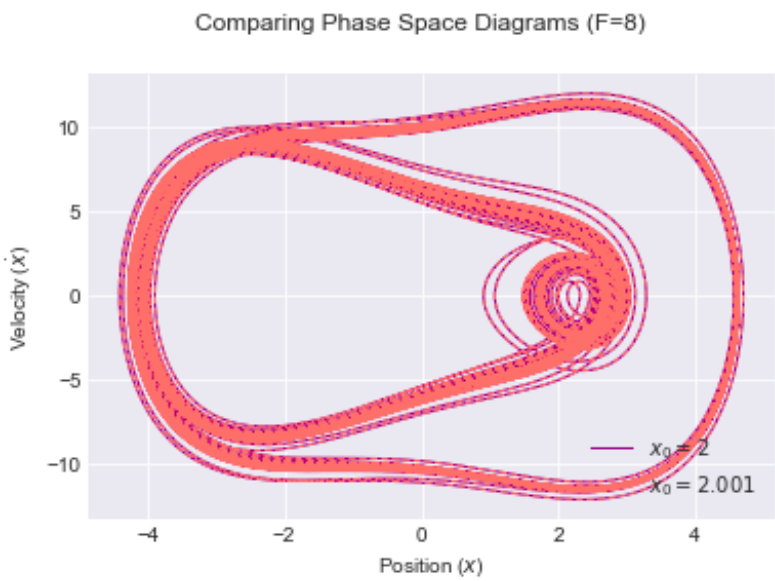

Figure 10: Comparison of the two stream plots, with the stream plots for $x = 2$ and $x = 2.001$ given by the colors purple and salmon respectively.

A Detour: The Liapunov Exponent

In general[11], the difference $\Delta\phi(t)$ between two identical driven damped pendulums (DDP) is given by[12]

$$\Delta\phi(t) \sim Ke^{\lambda t} \tag{19}$$

where K is a positive constant and λ is called the **Liapunov exponent**. In general, the sign of the Liapunov exponent tells us what kind of long-term motion a particular pendulum will display. If motion is chaotic (erratic, "unpredictable", nonperiodic), then the exponent will be positive. If the motion is nonchaotic, then the exponent will be negative. ■

In order to solve for the Liapunov Exponent for our particular problem, we can take (19) and modify it to fit our driven dampened oscillator: thus,

$$\Delta x(t) \sim Ke^{\lambda t} \tag{20}$$

and, solving for λ,

$$LN(\Delta x(t)) \sim \lambda t \tag{21}$$

$$\lambda \sim \frac{LN(\Delta x(t))}{t} \tag{22}$$

Numerically, we can plug this into python, as given by Figure 13. In Figure 13, we take the natural log of the absolute difference between the solutions ($\Delta x(t)$), which results in the list of numbers as given in Figure 14. We predict from the list, based on the sign in front of all of the terms, that we might expect chaotic motion due to the fact that all of the terms are positive; however, we can also average all of these terms to eliminate guessing from the equation altogether (since there are physically many many terms). Thus, the next line of code generates the average of all of these values, as coded in Figure 13 and as generated in Figure 14. Due to the fact that this exponent is positive, we can expect to see chaotic long-term behavior for this driven dampened oscillator. Notice, this numeric value matches our prediction in the previous part.

---

[11] Taylor, J. R. (2005). Classical Mechanics. University Science Book

[12] The $\sim$ signifies that $\Delta\phi(t)$ may oscillate underneath the right-hand side of the equation.

```
f6 = plt.figure()
plt.suptitle('Differences Between Solutions on Logarithmic Scale')
plt.xlabel('Time ($t$)')
plt.ylabel('Logarithmic Difference Between Solutions')
plt.plot(ts,np.log(np.absolute(sols1[:,1]-sols[:,0])),color='gray')
plt.xlim(0,200)
plt.show()
```

Figure 11: Python code, plotting time vs. the natural log of the absolute value of the difference (as given in equation (18)) between our two previous solutions. Output of this code is given in Figure 12.

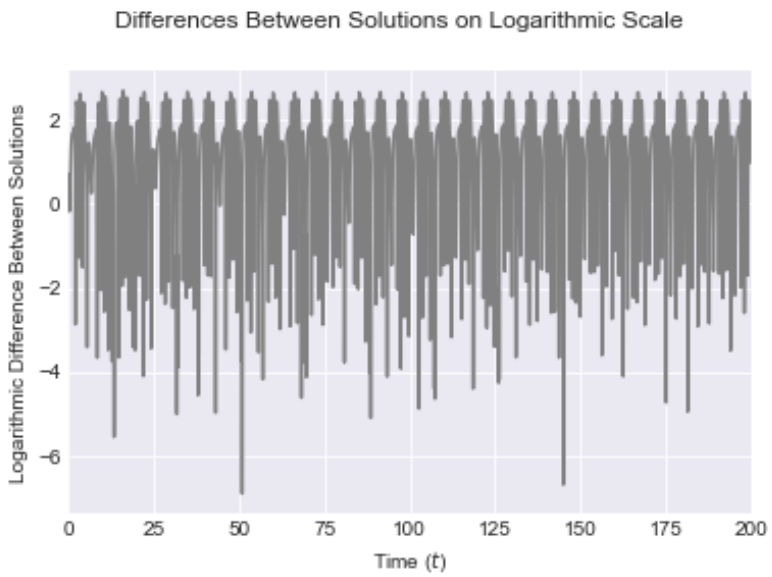

Figure 12: Generated graph, of time vs. the natural log of the absolute value of the differences between our solutions, from the python code as given in Figure 11. This is the generated graph given that $\omega = 2$.

CHANGING THE FREQUENCY OF OSCILLATIONS ($\omega = 1.2$)

(c) **Repeat part (b) but now with $\omega = 1.2$.**
Fortunately, this part will be very straightforward. Recall (17),

$$\frac{dv}{dt} = -0.05v - (-3)x + x^3 + 8cos(2t)$$

which was given in terms of $\omega = 2$. Thus, if we desire to do the same exact things but for $\omega = 1.2$, all we have to do is repeat our work but substitute in this new ω value. I neglect to show the repeated code for this, but I do include the new Phase Space Diagrams, the new differences between the two solutions, and the newly generated Liapunov Exponent, as given by Figures 15-19.

```
leconstants = np.log(np.absolute(sols1[:,1]-sols[:,0]))[1:-1]/ts[1:-1]
print(leconstants)
print(np.average(leconstants))
```

Figure 13: Python code calculating (1) a list of Liapunov Exponents and (2) the average of those exponents.

```
[6.62904080e+01 3.16048928e+01 2.00250288e+01 ... 5.35411893e-03
 5.18916839e-03 5.01680644e-03]
0.052254778674133404
```

Figure 14: Output values from the Python code as located in Figure 13: (1) a list of Liapunov Exponents and (2) the average of those exponents. The positive sign on each of these terms indicates that the motion will see will be chaotic.

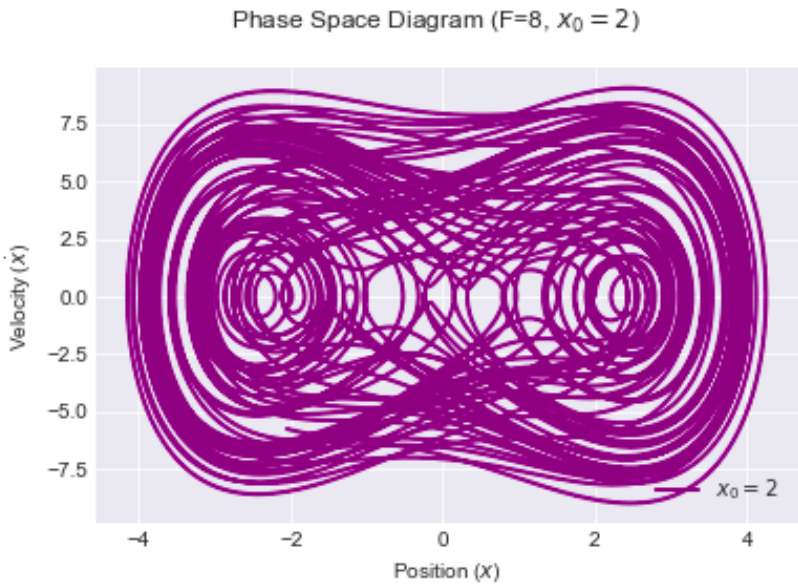

Figure 15: Phase space diagram, with $x_0 = 2$, $\omega = 1.2$, and $F = 8$.

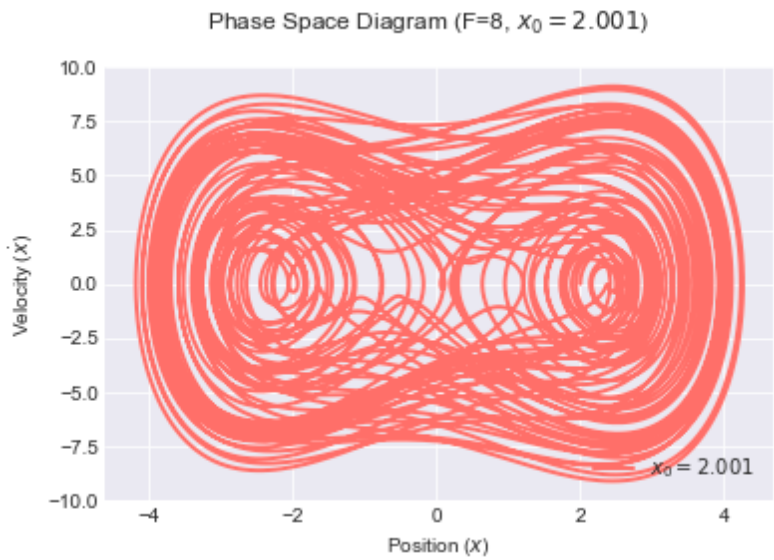

Figure 16: Phase space diagram, with $x_0 = 2.001$, $\omega = 1.2$, and $F = 8$.

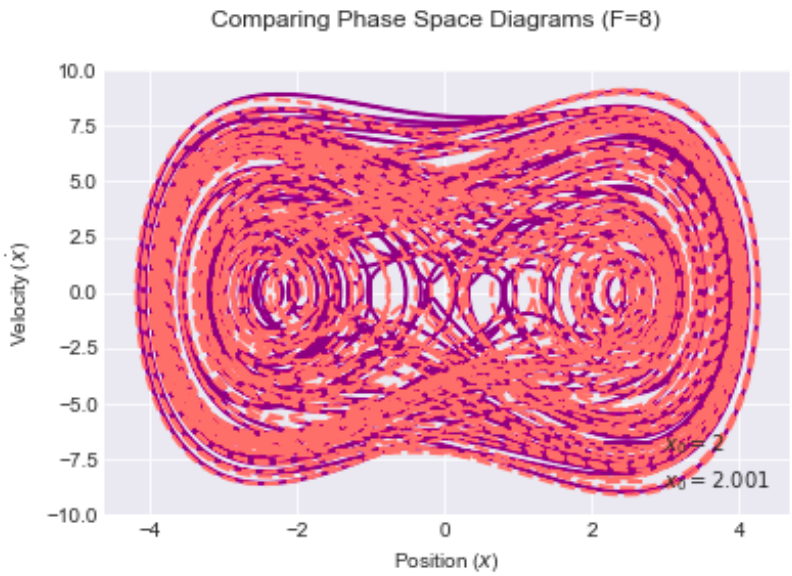

Figure 17: Comparison of the two stream plots, with the stream plots for $x = 2$ and $x = 2.001$ given by the colors purple and salmon respectively.

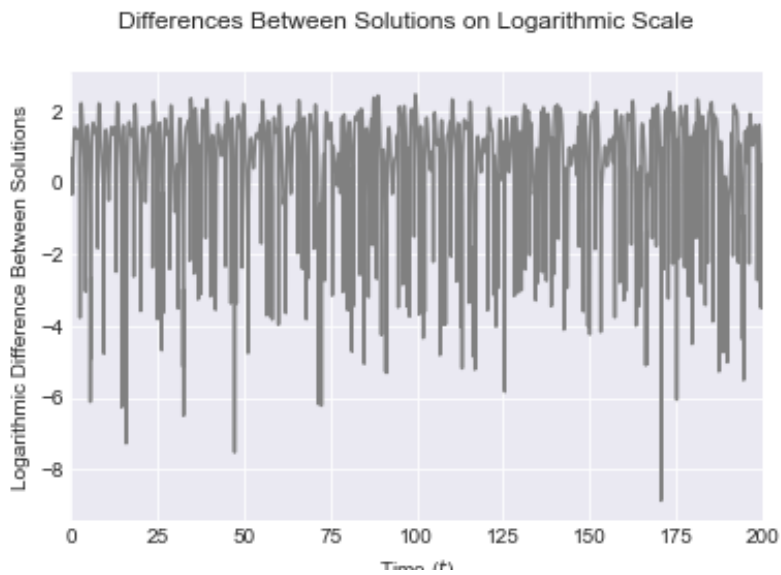

Figure 18: Generated graph, of time vs. the natural log of the absolute value of the differences between our solutions, given that $\omega = 1.2$.

```
[6.62902325e+01 3.16041711e+01 2.00233584e+01 ... 2.53924073e-03
 2.56340601e-03 2.58471660e-03]
0.03826407287897053
```

Figure 19: Output values for the same Python code as in Figure 13 but with $\omega = 1.2$. As previously, it lists (1) a list of Liapunov Exponents and (2) the average of those exponents. The positive sign on each of these terms indicates that the motion will see will be chaotic.

A POINCARE SECTION FOR $\omega = 1.2$

(d) **Create a Poincare section for the same parameters as given in part (c). This is mostly the same process as finding the solutions like you have been above, but you'll want to look at the time only once every period (so adjust your step size in times accordingly). In addition, you'll want to plot the phase space as points rather than lines. The minimum time to run the solution over to see a decent strange attractor is 10000 periods. Be patient, it will take a bit of time to process this. Moreover, since you only want the long term motion, you should really ignore the first 500 or so iterations.**

A Detour: Poincare Sections Earlier we created a phase space diagram simplify the trajectory our driver damped oscillator would take. A Poincare Section[13] is an even further simplification: it is a bunch of subsections of our previous data. Rather than plotting the "position" (in the form of a particular ordered pair (x, v), as seen in the phase space diagrams) of the oscillator, it is possible, using a Poincare Section, to plot the oscillator after the ending of every time period. That is, we can take a list t such that

$$t = \{t_0 + n | n \in \mathbb{N} \cup \{0\}\} \tag{23}$$

for some time period t_0. Without the "math lingo" involved, our list of times/cross-sections we will be considering is the set

$$t = \{t_0, t_0 + 1, ..., t_0 + n\} \tag{24}$$

for some[14] integer n. Additionally, when the Poincare section of the motion of a chaotic system is a fractal (aka those really cool pictures that when you keep zooming in keep looking like the same picture), the long-term motion is said to be a strange attractor. ■

For us, in order to see any kind of decent strange attractor, we will have to loop over many time periods. I consider the same time period we have already been working with (from 0 to 200 seconds); however, because we are only interested in the chaotic behavior (i.e. in order to see a decent strange attractor), I ignore the first 500 seconds and only plot a time interval from 500 to 200000 seconds (nearly 10000 time periods!).

```
newts = np.arange(500, 200000, 1)

sols2 = odeint(DE,state1,newts)

#Graphing time~
f2 = plt.figure()
#Titling and Coloring :)
plt.suptitle('Poincare Section ($\omega=1.2$)')
#Since this is a phase space diagram, plot position vs. velocity
plt.plot(sols2[:,0],sols2[:,1], 'o', ms=0.1, color='purple')
plt.xlabel('Position ($x$)')
plt.ylabel('Velocity ($\dot{x}$)')
plt.savefig('COOL.png')
plt.show()
```

Figure 20: Python code generating the Poincare Section, in Figure 21, for $\omega = 1.2$.

[13] Taylor, J. R. (2005). Classical Mechanics. University Science Book
[14] Rip. For some reason the integer sign wouldn't work

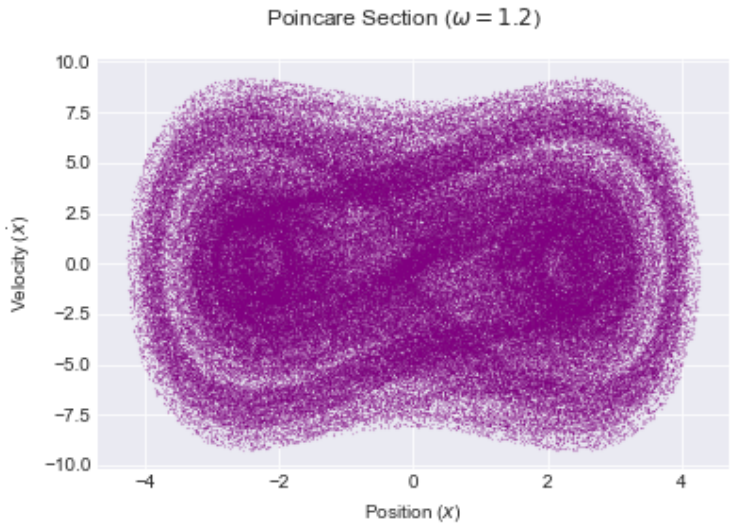

Figure 21: Generated Poincare Section, with Python Code located in Figure 20, for $\omega = 1.2$.

CONCLUSION

From just the snippet of behavior we have gathered in the past couple lines of coding, we can see the really interesting behavior that we can get from a simple ODE with a non-linear restoring force. In particular, we saw that, by changing the amplitude of the driving force (γ), the behavior of our phase space diagrams varied dramatically! Additionally, by changing ω even slightly, as we did in part b, the dramatic changes in our phase-space diagrams and the evolving long-term behavior of the duffing oscillator illustrates just how sensitive it can be toward varying initial conditions[15].

[15] Guess what: There's so much of this paper to still be expanded on! It was pushing to be very long as it was (already at 14 pages), so I go through the bare bones of solving the problem and providing background information. I try to elaborate where it's cool, but I fear much more elaboration would make my fellow peers want to murder me for taking up all of the peer evaluation time... In particular, I especially wanted to elaborate on the "peaks" as given in the graphs of the differences between the solutions based logarithmically/the implications of the "peaks". But that would (1) not only expand the paper, but also (2) involve zooming in on particular intervals.